

1. IMPLEMENT THE LIST ADT USING ARRAYS AND LINKED LISTS

AIM:

To write a c++ program to implement the List ADT using arrays and linked lists.

ALGORITHM:

1. Start the program.
2. Create a class ArrayList and declare *arr, size and capacity as class variables.
3. Create ArrayList constructor function and initialize the class variables.
4. Create ~ArrayList destructor function for delete the class variable.
5. Create member functions in ArrayList class called,
 - insert(val) – for insert a element using arr[size] and increase size value by one.
 - remove(val) – for delete the existing element using arr[] and shift the next elements.
 - display() – for printing the elements in the arr[]
6. Create a next class Node and declare data, *next, then create constructor function for initialize the class variables.
7. Create a next class LinkedList and declare *head, then create constructor function for initialize the class variable.
8. Create the member functions in LinkedList class called,
 - insert(val) – for create and insert a new element in a list.
 - remove(val) – for search and delete the element in a list.
 - display() – for display the elements in a list.
9. Create a main function.
10. Create an object arrList(5) and call the member functions from ArrayList class.
11. Create an object linkedList and call the member functions from LinkedList class.
12. Execute the program and print the result.
13. Stop the program.

SOURCE CODE:

```
#include <iostream.h>
#include <conio.h>
```

class ArrayList

```
{
    int* arr;
    int size;
    int capacity;
public:
    ArrayList(int cap)
    {
        capacity = cap;
        size = 0;
        arr = new int[capacity];
```

```

}
~ArrayList()
{
    delete[] arr;
}
void insert(int val)
{
    if (size < capacity)
    {
        arr[size++] = val;
    }
    else
    {
        cout << "Array is full!\n";
    }
}
void remove(int val)
{
    for (int i = 0; i < size; ++i)
    {
        if (arr[i] == val)
        {
            // Shift elements left
            for (int j = i; j < size - 1; ++j)
            {
                arr[j] = arr[j + 1];
            }
            size--;
            cout << "Element " << val << " removed.\n";
            return;
        }
    }
    cout << "Element " << val << " not found.\n";
}
void display()
{
    if (size == 0)
    {
        cout << "Array is empty.\n";
        return;
    }
    for (int i = 0; i < size; ++i) {
        cout << arr[i] << " ";
    }
    cout << endl;
}

```

```
};
```

```
class Node
```

```
{  
    public:  
        int data;  
        Node* next;  
        Node(int val)  
        {  
            data = val;  
            next = NULL;  
        }  
};
```

```
class LinkedList
```

```
{  
    Node* head;  
public:  
    LinkedList()  
    {  
        head = NULL;  
    }  
    void insert(int val)  
    {  
        Node* newNode = new Node(val);  
        newNode->next = head;  
        head = newNode;  
    }  
    void remove(int val)  
    {  
        if (head == NULL)  
        {  
            cout << "List is empty.\n";  
            return;  
        }  
        if (head->data == val)  
        {  
            Node* temp = head;  
            head = head->next;  
            delete temp;  
            cout << "Element " << val << " removed.\n";  
            return;  
        }  
        Node* current = head;  
        while (current->next != NULL && current->next->data != val)  
        {
```

```

        current = current->next;
    }
    if (current->next == NULL)
    {
        cout << "Element " << val << " not found.\n";
    }
    else
    {
        Node* temp = current->next;
        current->next = current->next->next;
        delete temp;
        cout << "Element " << val << " removed.\n";
    }
}
}
void display()
{
    if (head == NULL)
    {
        cout << "List is empty.\n";
        return;
    }
    Node* current = head;
    while (current != NULL)
    {
        cout << current->data << " ";
        current = current->next;
    }
    cout << endl;
}
};

```

```

void main()
{
    // Test ArrayList
    ArrayList arrList(5); // Initialize array with a capacity of 5
    clrscr();

    cout<<"\t\tList ADT implementation using Array"<<endl;
    cout<<"Array Operations:"<<endl;
    arrList.insert(10);
    arrList.insert(20);
    arrList.insert(30);
    arrList.display(); // Output: 10 20 30
    arrList.remove(20);
    arrList.display(); // Output: 10 30
}

```

```

arrList.remove(40); // Output: Element 40 not found.
cout<<"\n\n";

// Test LinkedList
LinkedList linkedList;
cout<<"\t\tList ADT implementation using Linked List"<<endl;
cout<<"Linked List Operations:"<<endl;
linkedList.insert(10);
linkedList.insert(20);
linkedList.insert(30);
linkedList.display(); // Output: 30 20 10
linkedList.remove(20);
linkedList.display(); // Output: 30 10
linkedList.remove(40); // Output: Element 40 not found.
getch();
}

```

OUTPUT:

```

DOSBox 0.74, Cpu speed: max 100% cycles, Frameskip 0, Program: TC
List ADT implementation using Array
Array Operations:
10 20 30
Element 20 removed.
10 30
Element 40 not found.

List ADT implementation using Linked List
Linked List Operations:
30 20 10
Element 20 removed.
30 10
Element 40 not found.

```

RESULT:

The above c++ program for implement the list ADT using arrays and linked list is executed successfully.

2. IMPLEMENT THE STACK ADT AND QUEUE ADT USING SINGLY LINKED LIST

AIM:

To write a C++ Program to implement the Stack ADT and Queue ADT using singly linked list.

ALGORITHMS:

1. Start the program.
2. Create a class Node and declare data, *next class variables then create constructor function for initialize the class variables.
3. Create a class Stack with *top then create constructor function for initialize the top as NULL.
4. Create the member functions in Stack class called,
 - ✓ push(val) – To insert the node value in to the linked list.
 - ✓ pop() - To delete the existing node in the linked list.
 - ✓ display() - To display the list of elements in the linked list.
5. Create a class Queue and declare *front and *rear class variables then create constructor function for initialize the top as NULL.
6. Create the member functions in Queue class called,
 - ✓ enqueue(val) – To insert the node value in to the linked list.
 - ✓ dequeue() - To delete the existing node in the linked list.
 - ✓ display() - To display the list of elements in the linked list.
7. Create the main function and create the objects for Stack and Queue classes.
8. Call the member functions from Stack class with Stack object.
9. Call the member functions from Queue class with Queue object.
10. Execute the program and display the result.
11. Stop the program.

SOURCE CODE:

```
#include <iostream.h>
#include <conio.h>
```

```
class Node
```

```
{
public:
    int data;
    Node* next;
```

```
    Node(int value)
```

```
    {
        data = value;
        next = NULL;
    }
};
```

```
class Stack
```

```

{
private:
    Node* top;

public:
    Stack()
    {
        top = NULL;
    }

    void push(int value)
    {
        Node* newNode = new Node(value);
        newNode->next = top;
        top = newNode;
        cout << "Pushed " << value << " to stack." << endl;
    }

    void pop()
    {
        if (top==NULL)
        {
            cout << "Stack Underflow!" << endl;
            return;
        }
        Node* temp = top;
        cout << "Popped " << top->data << " from stack." << endl;
        top = top->next;
        delete temp;
    }

    void display()
    {
        cout << "Stack (top to bottom): \n";
        Node* current = top;
        while (current)
        {
            cout << current->data << endl;
            current = current->next;
        }
    }
};

class Queue
{
private:

```

```
Node* front;  
Node* rear;
```

```
public:
```

```
Queue()
```

```
{  
    front = rear = NULL;  
}
```

```
void enqueue(int value)
```

```
{  
    Node* newNode = new Node(value);  
    if (rear == NULL)  
    {  
        front = rear = newNode;  
    }  
    else  
    {  
        rear->next = newNode;  
        rear = newNode;  
    }  
    cout << "Enqueued " << value << " to queue." << endl;  
}
```

```
void dequeue()
```

```
{  
    if (front == NULL)  
    {  
        cout << "Queue Underflow!" << endl;  
        return;  
    }  
    Node* temp = front;  
    cout << "Dequeued " << front->data << " from queue." << endl;  
    front = front->next;  
    if (front == NULL)  
        rear = NULL;  
    delete temp;  
}
```

```
void display()
```

```
{  
    cout << "Queue (front to rear): \n";  
    Node* current = front;  
    while (current)  
    {  
        cout << current->data << endl;
```

```

        current = current->next;
    }
}
};

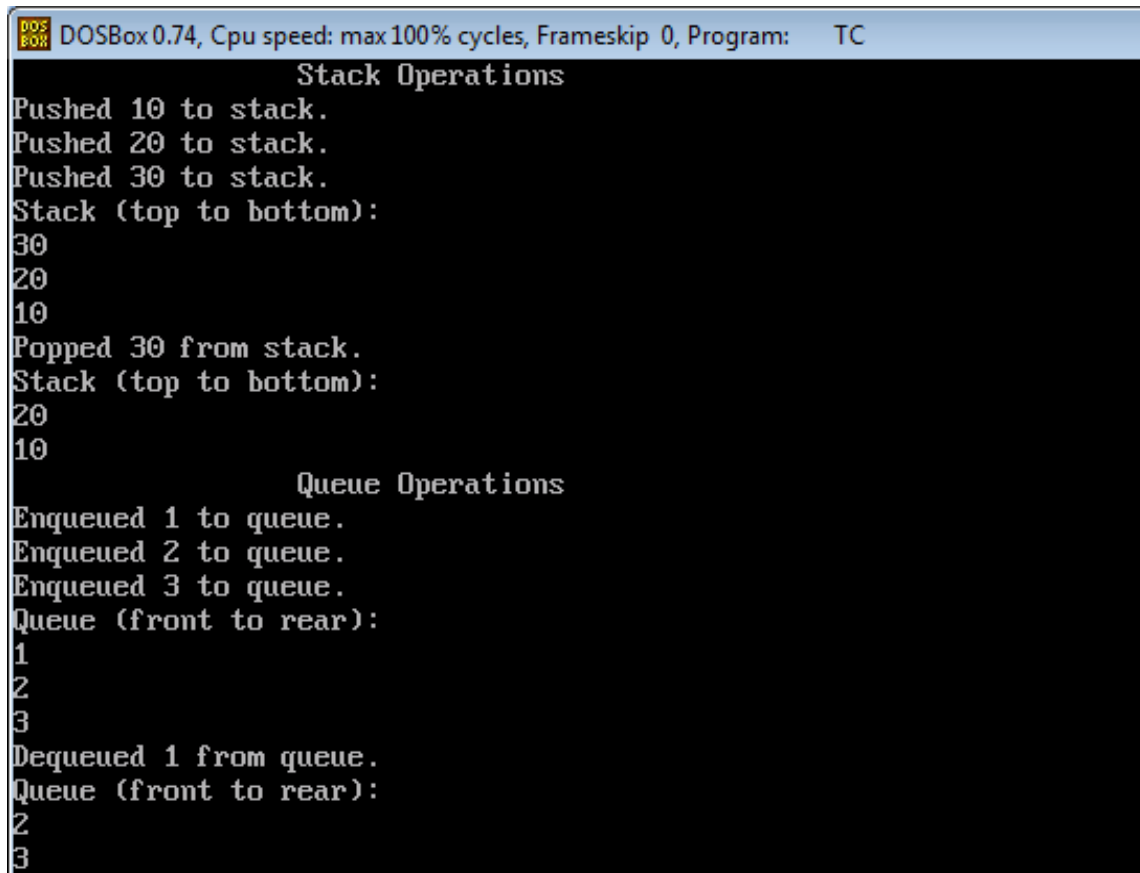
int main()
{
    Stack s;
    clrscr();
    cout<<"\t\t Stack Operations"<<endl;

    s.push(10);
    s.push(20);
    s.push(30);
    s.display();
    s.pop();
    s.display();

    Queue q;
    cout<<"\t\t Queue Operations"<<endl;
    q.enqueue(1);
    q.enqueue(2);
    q.enqueue(3);
    q.display();
    q.dequeue();
    q.display();
    getch();
    return 0;
}

```

OUTPUT:



```
DOSBox 0.74, Cpu speed: max 100% cycles, Frameskip 0, Program: TC

      Stack Operations
Pushed 10 to stack.
Pushed 20 to stack.
Pushed 30 to stack.
Stack (top to bottom):
30
20
10
Popped 30 from stack.
Stack (top to bottom):
20
10

      Queue Operations
Enqueued 1 to queue.
Enqueued 2 to queue.
Enqueued 3 to queue.
Queue (front to rear):
1
2
3
Dequeued 1 from queue.
Queue (front to rear):
2
3
```

RESULT:

The above c++ program for implement the Stack ADT and Queue ADT using singly linked list is executed successfully.

3. CONVERTS THE EXPRESSION TO POSTFIX EXPRESSION

AIM:

To read the infix expression and converts the infix expression to postfix express and evaluate the postfix expression.

ALGORITHM:

1. Start the program.
2. Define a function infixToPostfix(), for read the expression and check the each character,
 - a) If the character is '(' push() character in to the stack.
 - b) If the character is any operator check the precedence() and push() the character into the stack.
 - c) If the operator inside the stack have top precedence() to the incoming character, perform pop() operation.
 - d) If the operator is ')' pop() the operators from stack until reach '(' operator.
3. Define a function called precedence(), for fix the precedence of the operators.
4. Define a function called evaluatePostfix(), for evaluate the postfix expression and display the result.
5. Call the above all function as we need into the main function and get the result.
6. Stop the program.

SOURCE CODE:

```
#include <iostream>

#include <stack>

#include <cctype>

using namespace std;

// Get operator precedence

int precedence(char op)

{

    if(op == '+' || op == '-')

        return 1;

    if(op == '*' || op == '/')

        return 2;

    return 0;

}
```

// Convert infix to postfix

string infixToPostfix(string expr)

{

stack<char> st;

string postfix = "";

for (int i = 0; i < expr.length(); i++)

{

char c = expr[i];

if (isdigit(c))

{

postfix += c;

}

else if (c == '(')

{

st.push(c);

}

else if (c == ')')

{

while (!st.empty() && st.top() != '(')

{

postfix += st.top();

st.pop();

}

if (!st.empty()) st.pop();

// Remove '('

```

    }
    else
    {    // Operator
        while (!st.empty() && precedence(st.top()) >= precedence(c))
        {
            postfix += st.top();
            st.pop();
        }
        st.push(c);
    }
}

while (!st.empty())
{
    postfix += st.top();
    st.pop();
}

return postfix;
}

// Evaluate postfix expression
int evaluatePostfix(string postfix)
{
    stack<int> st;

    for (int i = 0; i < postfix.length(); i++)

```

```

{
    char c = postfix[i];

    if (isdigit(c))
    {
        st.push(c - '0');           // Convert char to int
    }
    else
    {
        int b = st.top();
        st.pop();
        int a = st.top();
        st.pop();
        switch (c) {
            case '+':
                st.push(a + b);
                break;
            case '-':
                st.push(a - b);
                break;
            case '*':
                st.push(a * b);
                break;
            case '/':
                st.push(a / b);
                break;
        }
    }
}

```

```
    }  
}  
  
    return st.top();  
}  
  
int main()  
{  
    string infix;  
    cout << "Enter infix expression (single-digit numbers only): ";  
    cin >> infix;  
  
    string postfix = infixToPostfix(infix);  
    cout << "Postfix: " << postfix << endl;  
  
    int result = evaluatePostfix(postfix);  
    cout << "Result: " << result << endl;  
  
    return 0;  
}
```

OUTPUT:

Enter infix expression (single-digit numbers only): (3+5)*2

Postfix: 35+2*

Result: 16

RESULT:

The above c++ program for read the infix expression and converts the infix expression to postfix express and evaluate the postfix expression is executed successfully.

4. IMPLEMENTATION OF PRIORITY QUEUE ADT

AIM:

To write a program to implementing priority queue ADT.

ALGORITHM:

1. Start the program.
2. Create a class PriorityQueue with data, priority and size as class variables.
3. Define PriorityQueue() constructor function for initialize the variable size =0.
4. Create member functions in PriorityQueue class called,
 - a) insert(int, int) – To get the priority and data from the user and insert in to the array with proper position based on the priority.
 - b) deleteHighestPriority() - To delete the first priority element inside the array.
 - c) display() – To display the elements from the array.
5. Create a main function.
6. Create an object pq for the class PriorityQueue and call the member functions.
7. Stop the program.

SOURCE CODE:

```
#include <iostream>

using namespace std;

const int MAX = 100;

class PriorityQueue
{
private:
    int data[MAX];
    int priority[MAX];
    int size;
```

public:

PriorityQueue()

```
{  
    size = 0;  
}
```

void insert(int item, int prio)

// Insert element with priority

```
{  
    if (size >= MAX)  
    {  
        cout << "Queue is full!\n";  
        return;  
    }  
}
```

int i; // Find position to insert new element based on priority

for (i = size - 1; i >= 0 && prio > priority[i]; i--)

```
{  
    data[i + 1] = data[i];  
    priority[i + 1] = priority[i];  
}
```

data[i + 1] = item; // Insert at the correct position

priority[i + 1] = prio;

size++;

```
}
```

void deleteHighestPriority()

// Delete highest priority element

```
{
```

if (size == 0)

```
{
```

cout << "Queue is empty!\n";

```

        return;
    }
    cout << "Deleted: " << data[0] << " (Priority: " << priority[0] << ")\n";
    for (int i = 1; i < size; i++)                // Shift all elements to the left
    {
        data[i - 1] = data[i];
        priority[i - 1] = priority[i];
    }
    size--;
}

void display()                                // Display the queue
{
    if (size == 0)
    {
        cout << "Queue is empty!\n";
        return;
    }
    cout << "Priority Queue contents:\n";
    for (int i = 0; i < size; i++)
    {
        cout << data[i] << " (Priority: " << priority[i] << ")\n";
    }
}

};

void main()                                    // Main function to test PriorityQueue ADT
{

```

```
PriorityQueue pq;  
  
pq.insert(10, 2);  
pq.insert(30, 3);  
pq.insert(20, 1);  
  
pq.display();  
  
pq.deleteHighestPriority();  
pq.display();  
}
```

OUTPUT:

Priority Queue contents:

30 (Priority: 3)

10 (Priority: 2)

20 (Priority: 1)

Deleted: 30 (Priority: 3)

Priority Queue contents:

10 (Priority: 2)

20 (Priority: 1)

RESULT:

The above c++ program for implementing priority queue ADT is executed successfully.

5. BINARY SEARCH TREE OPERATIONS

AIM:

To write a program to perform insert an element, delete an element and search the element in the Binary search tree.

ALGORITHM:

1. Start the program.
2. Create a class called Node and declare class variable then define the constructor function to initialize the class variables.
3. Create a class called BST and declare the class variables then define the constructor function to initialize the class variables.
4. Create member functions in BST class called,
 - a) Insert (Node, int) – for inserting node in the Binary search tree.
 - b) findMin(Node) – for finding the min value of node in the tree.
 - c) deleteNode(Node, int) – for delete the node in the Binary search tree.
 - d) search(Node, int) – for search the element in the Binary search tree.
 - e) inorder(Node) – for display the node data in the in-order form.
5. Define the main function and create the object for BST class.
6. Get the input from the user for select the menu using do-while loop and call the function with proper parameters as they selected menu.
7. Stop the program.

SOURCE CODE:

```
#include <iostream>

using namespace std;
```

```
class Node {
public:
    int data;
    Node* left;
    Node* right;
```

```
Node(int value) {  
    data = value;  
    left = right = NULL;  
}  
};
```

```
class BST {
```

```
public:
```

```
Node* root;
```

```
BST() {
```

```
    root = NULL;
```

```
}
```

```
Node* insert(Node* node, int value)
```

```
{
```

```
    if (node == NULL)
```

```
        return new Node(value);
```

```
    if (value < node->data)
```

```
        node->left = insert(node->left, value);
```

```
    else if (value > node->data)
```

```
        node->right = insert(node->right, value);
```

```
    return node;
```

```
}
```

Node* findMin(Node* node)

```
{  
    while (node && node->left != NULL)  
        node = node->left;  
    return node;  
}
```

Node* deleteNode(Node* node, int value)

```
{  
    if (node == NULL)  
        return node;  
  
    if (value < node->data)  
        node->left = deleteNode(node->left, value);  
    else if (value > node->data)  
        node->right = deleteNode(node->right, value);  
    else {
```

// Node with one or no child

```
        if (node->left == NULL) {  
            Node* temp = node->right;  
            delete node;  
            return temp;  
        } else if (node->right == NULL) {  
            Node* temp = node->left;  
            delete node;  
            return temp;  
        }  
}
```

// Node with two children

```
Node* temp = findMin(node->right);
node->data = temp->data;
node->right = deleteNode(node->right, temp->data);
}
return node;
}
```

bool search(Node* node, int value)

```
{
    if (node == NULL)
        return false;
    if (node->data == value)
        return true;
    else if (value < node->data)
        return search(node->left, value);
    else
        return search(node->right, value);
}
```

void inorder(Node* node)

```
{
    if (node != NULL) {
        inorder(node->left);
        cout << node->data << " ";
        inorder(node->right);
    }
}
```

```
    }  
}  
};
```

```
int main()
```

```
{  
    BST tree;  
    int choice, value;
```

```
do {
```

```
    cout << "\n1. Insert\n2. Delete\n3. Search\n4. Display (Inorder)\n5. Exit\nEnter choice: ";  
    cin >> choice;
```

```
switch (choice) {
```

```
    case 1:
```

```
        cout << "Enter value to insert: ";  
        cin >> value;  
        tree.root = tree.insert(tree.root, value);  
        break;
```

```
    case 2:
```

```
        cout << "Enter value to delete: ";  
        cin >> value;  
        tree.root = tree.deleteNode(tree.root, value);  
        break;
```

```
    case 3:
```

```
        cout << "Enter value to search: ";
        cin >> value;
        if (tree.search(tree.root, value))
            cout << "Found\n";
        else
            cout << "Not Found\n";
        break;

    case 4:
        cout << "Inorder Traversal: ";
        tree.inorder(tree.root);
        cout << endl;
        break;

    case 5:
        cout << "Exiting...\n";
        break;

    default:
        cout << "Invalid choice\n";
    }
} while (choice != 5);

return 0;
}
```

OUTPUT:

1. Insert
2. Delete
3. Search
4. Display (Inorder)
5. Exit
Enter choice: 1
Enter value to insert: 45

1. Insert
2. Delete
3. Search
4. Display (Inorder)
5. Exit
Enter choice: 1
Enter value to insert: 56

1. Insert
2. Delete
3. Search
4. Display (Inorder)
5. Exit
Enter choice: 1
Enter value to insert: 6

1. Insert
2. Delete
3. Search
4. Display (Inorder)
5. Exit
Enter choice: 4
Inorder Traversal: 6 45 56

1. Insert
2. Delete
3. Search
4. Display (Inorder)
5. Exit
Enter choice: 3
Enter value to search: 45
Found

1. Insert
2. Delete
3. Search

4. Display (Inorder)
5. Exit
Enter choice: 3
Enter value to search: 77
Not Found

1. Insert
2. Delete
3. Search
4. Display (Inorder)
5. Exit
Enter choice: 5
Exiting...

RESULT:

The above c++ program for perform insert an element, delete an element and search the element in the Binary search tree is executed successfully.

6. AVL TREE OPERATIONS

AIM:

To write a program to perform the insertion and deletion operation in the AVL tree.

ALGORITHM:

1. Start the program.
2. Create a class Node and declare the variables called key, left, right and height.
3. Create the constructor function and initialize the class variables.
4. Create a class AVLTree and create the following function,
 - a) getHeight() – To return the height of the node.
 - b) getBalance() – To return the node balance.
 - c) rightRotate() – To rotate the tree in right side.
 - d) leftRotate() – To rotate the tree in left side.
 - e) insertNode() – To insert the new node in the tree.
 - f) deleteNode() – To delete the existing node in the tree.
 - g) inorderTraversal() – To perform the inorder traversal.
5. Create the main function and get the input from the user and perform the operation using AVLTree object called tree.

SOURCE CODE:

```
#include <iostream>
```

```
#include <algorithm>
```

```
using namespace std;
```

```
class Node
```

```
{
```

```
public:
```

```
    int key;
```

```
Node* left;
```

```
Node* right;
```

```
int height;
```

```
Node(int value)
```

```
{
```

```
    key = value;
```

```
    left = right = nullptr;
```

```
    height = 1;
```

```
}
```

```
};
```

```
class AVLTree
```

```
{
```

```
private:
```

```
    Node* root;
```

```
int getHeight(Node* node)
```

```
{
```

```
    return node ? node->height : 0;
```

```
}
```

```
int getBalance(Node* node)
```

```
{
```

```
    return node ? getHeight(node->left) - getHeight(node->right) : 0;
```

```
}
```

```
Node* rightRotate(Node* y)
```

```
{
```

```
    Node* x = y->left;
```

```

Node* T2 = x->right;
x->right = y;
y->left = T2;
y->height = 1 + max(getHeight(y->left), getHeight(y->right));
x->height = 1 + max(getHeight(x->left), getHeight(x->right));
return x;
}

```

Node* leftRotate(Node* x)

```

{
Node* y = x->right;
Node* T2 = y->left;

y->left = x;
x->right = T2;
x->height = 1 + max(getHeight(x->left), getHeight(x->right));
y->height = 1 + max(getHeight(y->left), getHeight(y->right));
return y;
}

```

Node* insertNode(Node* node, int key)

```

{
if (!node)
    return new Node(key);

if (key < node->key)
    node->left = insertNode(node->left, key);
else if (key > node->key)

```

```

        node->right = insertNode(node->right, key);
    else
        return node; // Duplicate keys not allowed

    node->height = 1 + max(getHeight(node->left), getHeight(node->right));
    int balance = getBalance(node);

    // Balance the node
    if (balance > 1 && key < node->left->key)
        return rightRotate(node);

    if (balance < -1 && key > node->right->key)
        return leftRotate(node);

    if (balance > 1 && key > node->left->key)
    {
        node->left = leftRotate(node->left);
        return rightRotate(node);
    }
    if (balance < -1 && key < node->right->key)
    {
        node->right = rightRotate(node->right);
        return leftRotate(node);
    }
    return node;
}

```

Node* minValueNode(Node* node)

```
{  
    Node* current = node;  
    while (current->left)  
        current = current->left;  
    return current;  
}
```

Node* deleteNode(Node* root, int key)

```
{  
    if (!root)  
        return root;  
  
    if (key < root->key)  
        root->left = deleteNode(root->left, key);  
    else if (key > root->key)  
        root->right = deleteNode(root->right, key);  
    else {  
        if (!root->left || !root->right)  
        {  
            Node* temp = root->left ? root->left : root->right;  
            if (!temp)  
            {  
                temp = root;  
                root = nullptr;  
            }  
            else
```

```

        *root = *temp;

        delete temp;
    }
    else
    {
        Node* temp = minValueNode(root->right);
        root->key = temp->key;
        root->right = deleteNode(root->right, temp->key);
    }
}

if (!root)
    return root;

root->height = 1 + max(getHeight(root->left), getHeight(root->right));
int balance = getBalance(root);

if (balance > 1 && getBalance(root->left) >= 0)
    return rightRotate(root);

if (balance > 1 && getBalance(root->left) < 0)
{
    root->left = leftRotate(root->left);
    return rightRotate(root);
}

```

```

        if (balance < -1 && getBalance(root->right) <= 0)
            return leftRotate(root);

        if (balance < -1 && getBalance(root->right) > 0)
        {
            root->right = rightRotate(root->right);
            return leftRotate(root);
        }
        return root;
    }

```

void inorderTraversal(Node* node)

```

{
    if (node)
    {
        inorderTraversal(node->left);
        cout << node->key << " ";
        inorderTraversal(node->right);
    }
}

```

public:

AVLTree()

```

{
    root = nullptr;
}

```

void insert(int key)

```

{
    root = insertNode(root, key);
}

```

```

    }

    void remove(int key)
    {
        root = deleteNode(root, key);
    }

    void inorder()
    {
        cout << "In-order Traversal: ";
        inorderTraversal(root);
        cout << endl;
    }
};

int main()
{
    AVLTree tree;
    int choice, value;

    do
    {
        cout << "\n----- AVL Tree Menu ----- \n";
        cout << "1. Insert\n";
        cout << "2. Delete\n";
        cout << "3. Display (In-order)\n";
        cout << "4. Exit\n";
        cout << "Enter your choice: ";
        cin >> choice;
    }

```

switch (choice)

{

case 1:

cout << "Enter value to insert: ";

cin >> value;

tree.insert(value);

cout << "Inserted " << value << " successfully.\n";

break;

case 2:

cout << "Enter value to delete: ";

cin >> value;

tree.remove(value);

cout << "Deleted " << value << " successfully.\n";

break;

case 3:

tree.inorder();

break;

case 4:

cout << "Exiting program.\n";

break;

default:

cout << "Invalid choice! Try again.\n";

```
    }  
    } while (choice != 4);  
    return 0;  
}
```

OUTPUT:

----- AVL Tree Menu -----

```
1. Insert  
2. Delete  
3. Display (In-order)  
4. Exit  
Enter your choice: 1  
Enter value to insert: 30  
Inserted 30 successfully.
```

----- AVL Tree Menu -----

```
1. Insert  
2. Delete  
3. Display (In-order)  
4. Exit  
Enter your choice: 1  
Enter value to insert: 20  
Inserted 20 successfully.
```

----- AVL Tree Menu -----

```
1. Insert  
2. Delete  
3. Display (In-order)  
4. Exit  
Enter your choice: 1  
Enter value to insert: 10  
Inserted 10 successfully.
```

----- AVL Tree Menu -----

```
1. Insert  
2. Delete  
3. Display (In-order)  
4. Exit  
Enter your choice: 3  
In-order Traversal: 10 20 30
```

----- AVL Tree Menu -----

1. Insert
2. Delete
3. Display (In-order)
4. Exit
Enter your choice: 2
Enter value to delete: 20
Deleted 20 successfully.

----- AVL Tree Menu -----

1. Insert
2. Delete
3. Display (In-order)
4. Exit
Enter your choice: 3
In-order Traversal: 10 30

----- AVL Tree Menu -----

1. Insert
2. Delete
3. Display (In-order)
4. Exit
Enter your choice: 4
Exiting program.

RESULT:

The above c++ program to perform the insertion and deletion operation in the AVL tree is executed successfully.

7. IMPLEMENTATION OF BFS AND DFS TRAVERSAL IN GRAPH

AIM:

To write a program to implement the BFS and DFS for a given graph.

ALGORITHM:

1. Start the program.
2. Create a class called Graph and declare the class variables and constructor for initialize the class variable.
3. Create the functions in the Graph class,
 - a) addEdge() – To add the edge between the nodes.
 - b) BFS() – To perform the Breadth First Traversal.
 - c) DFSUtil() – To perform Depth First Traversal until the end of node.
 - d) DFS() – To perform the Depth First Traversal.
4. Create the main function and perform add edges and traversal of BFS and DFS.
5. Stop the Program.

SOURCE CODE:

```
#include <iostream>

#include <vector>

#include <queue>

using namespace std;

class Graph
{
    int V;                                // Number of vertices
    vector<int> adj[100];                  // Adjacency list (fixed size for simplicity)

public:
    Graph(int v)
    {
        V = v;
    }
}
```

```

void addEdge(int u, int v)                                // Add edge (undirected)
{
    adj[u].push_back(v);
    adj[v].push_back(u);
}

void BFS(int start)                                       // BFS traversal
{
    bool visited[100] = {false};
    queue<int> q;
    visited[start] = true;
    q.push(start);
    cout << "BFS Traversal: ";
    while (!q.empty())
    {
        int node = q.front();
        q.pop();
        cout << node << " ";
        for (int i = 0; i < adj[node].size(); i++)
        {
            int neighbor = adj[node][i];
            if (!visited[neighbor])
            {
                visited[neighbor] = true;
                q.push(neighbor);
            }
        }
    }
}

```

```
    cout << endl;
}
```

```
void DFSUtil(int node, bool visited[])                                // DFS helper function
{
    visited[node] = true;
    cout << node << " ";
    for (int i = 0; i < adj[node].size(); i++)
    {
        int neighbor = adj[node][i];
        if (!visited[neighbor])
        {
            DFSUtil(neighbor, visited);
        }
    }
}
```

```
void DFS(int start)                                                // DFS traversal
{
    bool visited[100] = {false};
    cout << "DFS Traversal: ";
    DFSUtil(start, visited);
    cout << endl;
}

};
```

```
int main()
{
    int V, E;

    cout << "Enter number of vertices: ";
    cin >> V;

    Graph g(V);

    cout << "Enter number of edges: ";
    cin >> E;

    cout << "Enter edges (u v):\n";
    for (int i = 0; i < E; i++)
    {
        int u, v;
        cin >> u >> v;
        g.addEdge(u, v);
    }

    int start;
    cout << "Enter starting node: ";
    cin >> start;

    g.BFS(start);
    g.DFS(start);

    return 0;
}
```

OUTPUT:

Enter number of vertices: 5

Enter number of edges: 4

Enter edges (u v):

0 1

0 2

1 3

1 4

Enter starting node: 0

BFS Traversal: 0 1 2 3 4

DFS Traversal: 0 1 3 4 2

RESULT:

The above c++ program to implement the BFS and DFS for a given graph is executed successfully.

8. LINEAR SEARCH AND BINARY SEARCH

AIM:

To write a program for implementing the linear search and binary search.

ALGORITHM:

1. Start the program
2. Create a function called `linearSearch()` with three parameters and receive the key element and return the search result.
3. Create a function called `binarySearch()` with three parameters and receive the key element and return the search result.
3. Create the main function and generate the list of elements and perform the search operation based on the user choice.
4. Produce the output.
5. Stop the program.

SOURCE CODE:

```
#include <iostream>

using namespace std;
```

```
int linearSearch(int arr[], int size, int key)           // Linear Search
{
    for (int i = 0; i < size; i++)
    {
        if (arr[i] == key)
            return i; // Found at index i
    }
    return -1; // Not found
}
```

```
int binarySearch(int arr[], int size, int key)           // Binary Search (array must be sorted)
```

```
{  
    int left = 0, right = size - 1;  
    while (left <= right) {  
        int mid = (left + right) / 2;  
  
        if (arr[mid] == key)  
            return mid;  
        else if (key < arr[mid])  
            right = mid - 1;  
        else  
            left = mid + 1;  
    }  
    return -1; // Not found  
}
```

```
int main()
```

```
{  
    int arr[100], n, key, choice;  
    cout << "Enter number of elements: ";  
    cin >> n;  
    cout << "Enter " << n << " elements:\n";  
    for (int i = 0; i < n; i++)  
        cin >> arr[i];  
    cout << "Enter the key to search: ";  
    cin >> key;  
    cout << "\nChoose search method:\n";  
    cout << "1. Linear Search\n";  
}
```

```
cout << "2. Binary Search (array must be sorted)\n";
cout << "Enter choice: ";
cin >> choice;
int result;
switch (choice)
{
    case 1:
        result = linearSearch(arr, n, key);
        break;
    case 2:
        result = binarySearch(arr, n, key);
        break;
    default:
        cout << "Invalid choice!\n";
        return 0;
}
if (result != -1)
    cout << "Element found at index " << result << ".\n";
else
    cout << "Element not found.\n";

return 0;
}
```

OUTPUT:

Enter number of elements: 5

Enter 5 elements:

10 20 30 40 50

Enter the key to search: 30

Choose search method:

1. Linear Search

2. Binary Search (array must be sorted)

Enter choice: 2

Element found at index 2.

RESULT:

The above c++ program for implementing the linear search and binary search is executed successfully.

9. IMPLEMENTING THE SORTING METHODS

AIM:

To write a program for implementing the Bubble sort, Selection sort, Insertion sort and Radix sort.

ALGORITHM:

1. Start the program.
2. Create the function called bubbleSort() and perform the sorting operation.
3. Create the function called selectionSort() and perform the sorting operation.
4. Create the function called insertionSort() and perform the sorting operation.
5. Create the function getMax() to get the maximum value in the array.
6. Create the function called countingSort() and perform the counting process of occurrences in the elements in array.
7. Create the function called radixSort() and perform the sorting operation.
8. Stop the program

SOURCE CODE:

```
#include <iostream>

#include <cmath>

using namespace std;

void bubbleSort(int arr[], int n)                                // Bubble Sort
{
    for (int i = 0; i < n - 1; i++)
    {
        for (int j = 0; j < n - i - 1; j++)
        {
            if (arr[j] > arr[j + 1])
```

```

        {
            int temp = arr[j];
            arr[j] = arr[j + 1];
            arr[j + 1] = temp;
        }
    }
}

```

void selectionSort(int arr[], int n) // Selection Sort

```

{
    for (int i = 0; i < n - 1; i++)
    {
        int minIndex = i;
        for (int j = i + 1; j < n; j++)
        {
            if (arr[j] < arr[minIndex])
                minIndex = j;
        }
        int temp = arr[i];
        arr[i] = arr[minIndex];
        arr[minIndex] = temp;
    }
}

```

void insertionSort(int arr[], int n) // Insertion Sort

```

{

```

```

for (int i = 1; i < n; i++)
{
    int current = arr[i];
    int j = i - 1;

    while (j >= 0 && arr[j] > current)           // Shift elements to the right
    {
        arr[j + 1] = arr[j];
        j--;
    }
    arr[j + 1] = current;
}

int getMax(int arr[], int n)                   // Get maximum value in array
{
    int max = arr[0];
    for (int i = 1; i < n; i++)
        if (arr[i] > max)
            max = arr[i];
    return max;
}

void countingSort(int arr[], int n, int exp)    // Counting sort used in radix sort
{
    int output[100];
    int count[10] = {0};

```

```

for (int i = 0; i < n; i++)                                // Count occurrences
    count[(arr[i] / exp) % 10]++;

for (int i = 1; i < 10; i++)                                // Cumulative count
    count[i] += count[i - 1];

for (int i = n - 1; i >= 0; i--)                            // Build output array (stable sort)
{
    int digit = (arr[i] / exp) % 10;
    output[count[digit] - 1] = arr[i];
    count[digit]--;
}

for (int i = 0; i < n; i++)                                // Copy to original array
    arr[i] = output[i];
}

void radixSort(int arr[], int n)                            // Radix Sort
{
    int max = getMax(arr, n);
    for (int exp = 1; max / exp > 0; exp *= 10)
        countingSort(arr, n, exp);
}

void displayArray(int arr[], int n)                            // Display array
{
    for (int i = 0; i < n; i++)
        cout << arr[i] << " ";
}

```

```

        cout << endl;
    }

    int main()                                // Main function
    {
        int arr[100], n, choice;

        cout << "Enter number of elements: ";
        cin >> n;

        cout << "Enter " << n << " elements:\n";
        for (int i = 0; i < n; i++)
            cin >> arr[i];

        cout << "\nChoose sorting method:\n";
        cout << "1. Bubble Sort\n2. Selection Sort\n3. Insertion Sort\n4. Radix Sort\n";
        cout << "Enter choice: ";
        cin >> choice;

        switch (choice)
        {
            case 1:
                bubbleSort(arr, n);
                break;
            case 2:
                selectionSort(arr, n);
                break;
            case 3:

```

```
        insertionSort(arr, n);
        break;
    case 4:
        radixSort(arr, n);
        break;
    default:
        cout << "Invalid choice!\n";
        return 0;
}

cout << "Sorted array: ";
displayArray(arr, n);

return 0;
}
```

OUTPUT:

Enter number of elements: 6
Enter 6 elements:
170 45 75 90 802 24

Choose sorting method:
1. Bubble Sort
2. Selection Sort
3. Insertion Sort
4. Radix Sort
Enter choice: 4

Sorted array: 24 45 75 90 170 802

Enter number of elements: 5
Enter 5 elements:
5 2 9 1 7

Choose sorting method:

1. Bubble Sort
2. Selection Sort
3. Insertion Sort
4. Radix Sort

Enter choice: 1

Sorted array: 1 2 5 7 9

RESULT:

The above c++ program for implementing the Bubble sort, Selection sort, Insertion sort and Radix sort executed successfully.